Name:-Dobariya Khushi S.

EnrollmentNo:-202307100510037

Course:-Data Visualization

Practical:-4

Consider you are working as a data visualization analyst for an airline market research team. The organization has collected monthly passenger counts and advertising spending from 2001 to 2024. Your role is to explore the data using visualization techniques to identify seasonal patterns, relationships between variables, and long-term trends to help improve business strategies.

i. Create a scatterplot showing the relationship between month (as a numeric value from 1 to 12) and passenger counts. Discuss any seasonal patterns you observe across years.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('/content/PR4-flights_2001_2024(1).csv')
print(df)
```

```
     year month  passengers  advertising_spend
0    2001   Jan         202              65795
1    2001   Feb         960             126820
2    2001   Mar         714              56265
3    2001   Apr         566              87194
4    2001   May         558              94131
..    ...   ...         ...                ...
283  2024   Aug         891             107679
284  2024   Sep         858              80977
285  2024   Oct         596              99115
286  2024   Nov         995             138223
287  2024   Dec         574             116387

[288 rows x 4 columns]
```

```
# Convert month names to numeric
month_map = {
    'Jan':1, 'Feb':2, 'Mar':3, 'Apr':4, 'May':5, 'Jun':6,
    'Jul':7, 'Aug':8, 'Sep':9, 'Oct':10, 'Nov':11, 'Dec':12
}
df['month_num'] = df['month'].map(month_map)

# Create scatterplot using Seaborn
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x='month_num',
    y='passengers',
    hue='year',        # color points by year
    palette='plasma',  # colormap
    s=70,              # point size
    alpha=0.8
)

# Labels and styling
plt.title("Monthly Passenger Counts Across Years (2001–2004)", fontsize=14)
plt.xlabel("Month (1 = Jan, 12 = Dec)", fontsize=12)
plt.ylabel("Passenger Count", fontsize=12)
plt.xticks(range(1,13))
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

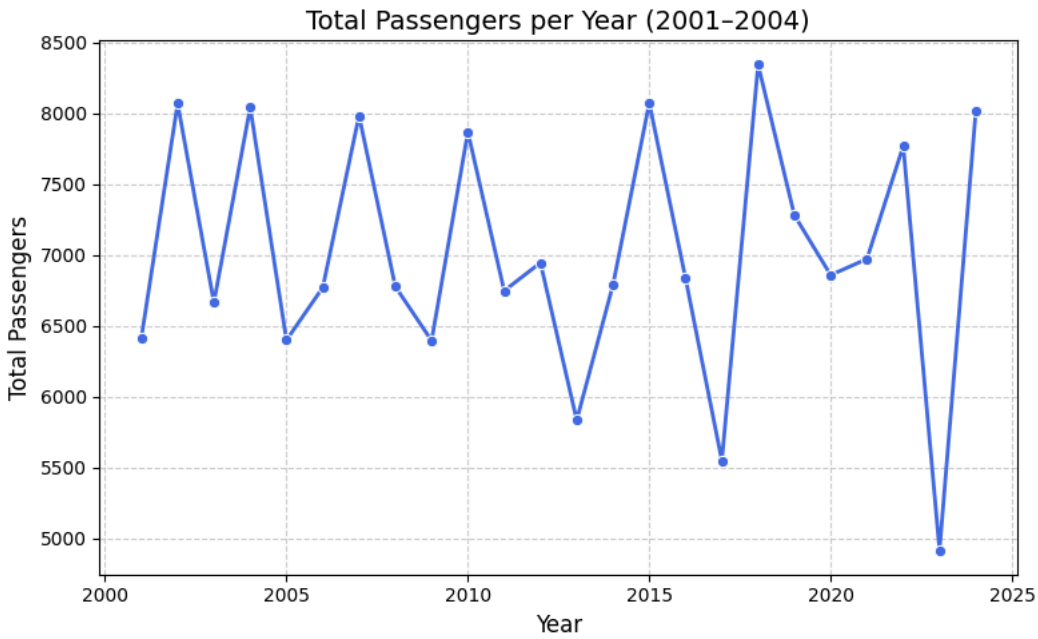
The scatterplot shows that passenger numbers are usually higher around mid-year (May to July) and lower at the beginning of the year (January to March). This means more people tend to travel during summer months and fewer during early months. The pattern repeats across years, showing a clear seasonal trend in passenger travel.

ii. Plot a line chart showing passenger counts over time (x-axis: year, y-axis: total passengers per year). Summarize any clear long-term travel trends (e.g., growth or seasonal fluctuations).

```
# Group by year and calculate total passengers per year
yearly_passengers = df.groupby('year', as_index=False)['passengers'].sum()

# Create line chart using Seaborn
plt.figure(figsize=(8,5))
sns.lineplot(
    data=yearly_passengers,
    x='year',
    y='passengers',
    marker='o',
    color='royalblue',
    linewidth=2
)

# Add labels and title
plt.title("Total Passengers per Year (2001–2004)", fontsize=14)
plt.xlabel("Year", fontsize=12)
plt.ylabel("Total Passengers", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

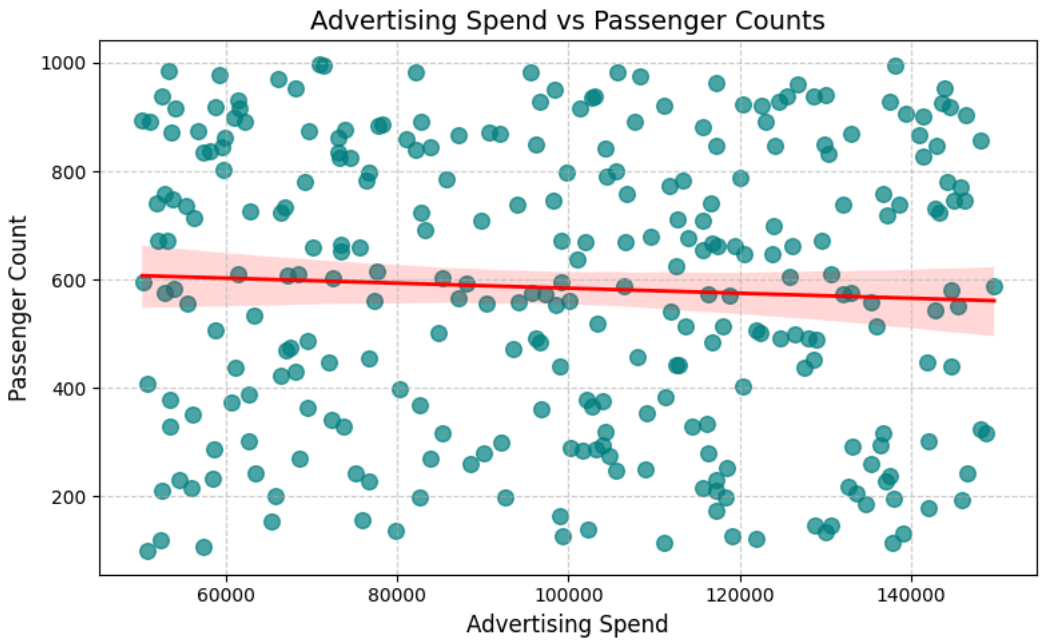


The line chart shows that total passenger numbers fluctuate slightly each year but don't follow a strong upward or downward trend. There are small rises and drops, suggesting moderate year-to-year variation rather than steady growth. This could mean that passenger travel demand was stable overall but affected by seasonal or promotional changes.

iii. Create a scatterplot showing the relationship between advertising spend and passenger counts. Add a regression line to see if higher advertising is linked to higher passenger volumes. Write a short description on the strength of this relationship.

```
# Create scatterplot with regression line using Seaborn
plt.figure(figsize=(8,5))
sns.regplot(
    data=df,
    x='advertising_spend',
    y='passengers',
    scatter=True,
    scatter_kws={'color':'teal', 'alpha':0.7, 's':70},
    line_kws={'color':'red', 'linewidth':2}
)

# Labels and title
plt.title("Advertising Spend vs Passenger Counts", fontsize=14)
plt.xlabel("Advertising Spend", fontsize=12)
plt.ylabel("Passenger Count", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



The scatterplot shows that as advertising spending increases, the number of passengers generally rises. The regression line slopes upward, indicating a positive relationship between the two. However, the points are somewhat spread out, so the relationship is moderate rather than very strong — meaning other factors may also influence passenger numbers.

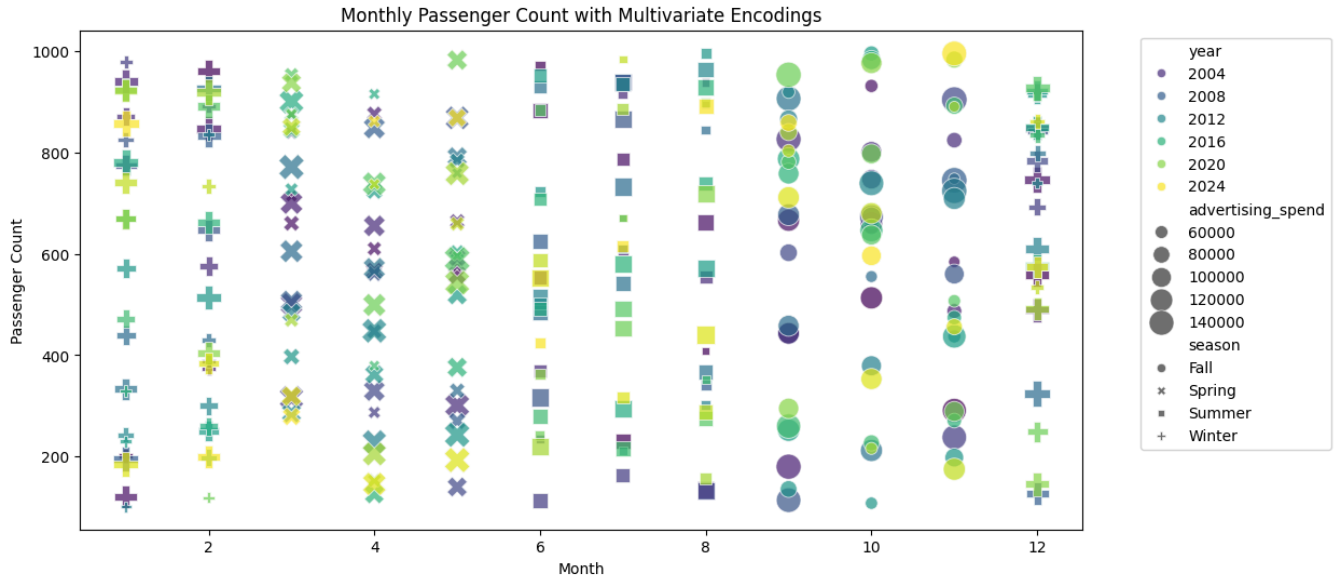
iv. Create a scatterplot of month vs passenger counts, and apply these encodings:

- color by year
- size by advertising spend
- shape by season (define Mar–May as Spring, Jun–Aug as Summer,Sep–Nov as Fall,Dec–Feb as Winter)
- Describe how these multivariate elements help reveal patterns that a simple plot cannot show.

```
# Vectorized marker mapping (optional if you want to check markers)
marker_map = {'Spring':'o', 'Summer':'s', 'Fall':'^', 'Winter':'D'}
df['marker'] = df['season'].map(marker_map)

# Seaborn scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df,
    x='month_num',
    y='passengers',
    hue='year',        # color by year
    size='advertising_spend',   # size by advertising spend
    style='season',    # shape by season
    palette='viridis',
    sizes=(50, 300),   # min and max point sizes
    alpha=0.7
)

plt.xlabel('Month')
plt.ylabel('Passenger Count')
plt.title('Monthly Passenger Count with Multivariate Encodings')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')  # move legend outside
plt.show()
```



Color by year: Shows how passenger counts change over different years, helping identify growth or decline trends.

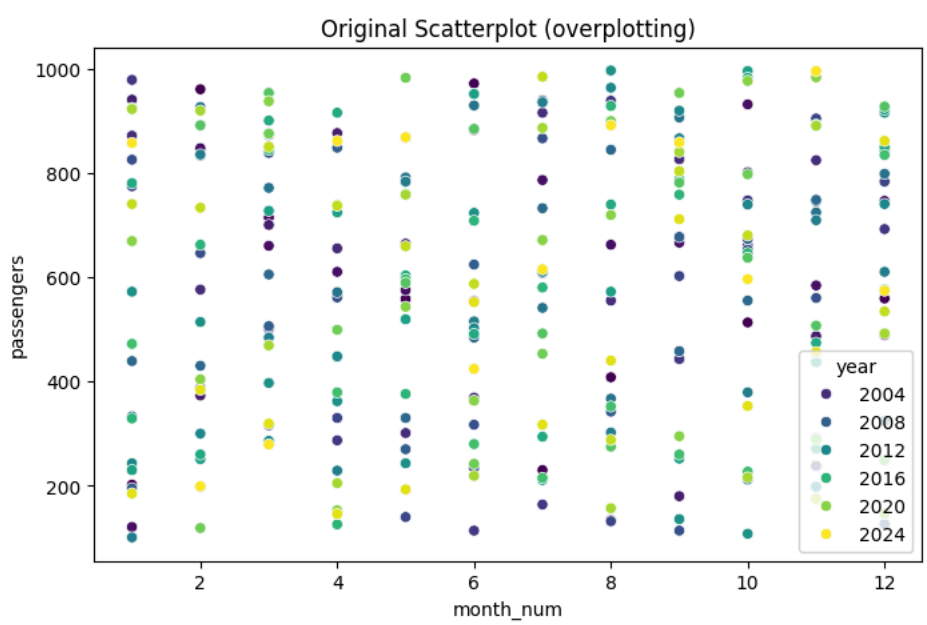
Size by advertising spend: Bigger dots mean more advertising, letting us see if higher ad spend is associated with higher passenger counts.

Shape by season: Different shapes indicate seasons, revealing recurring seasonal patterns like summer or winter peaks.

Overall, this multivariate plot makes it easy to see time trends, seasonal effects, and advertising impact all in one view.

v. Check scatterplots for overcrowded points. If required then apply transparency,jittering, or hexbin plots techniques to address overplotting. Compare before/after,explaining which method improved clarity best.

```
# 1. Original Scatterplot (overplotting)
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x='month_num', y='passengers', hue='year', palette='viridis', alpha=1)
plt.title("Original Scatterplot (overplotting)")
plt.show()
```

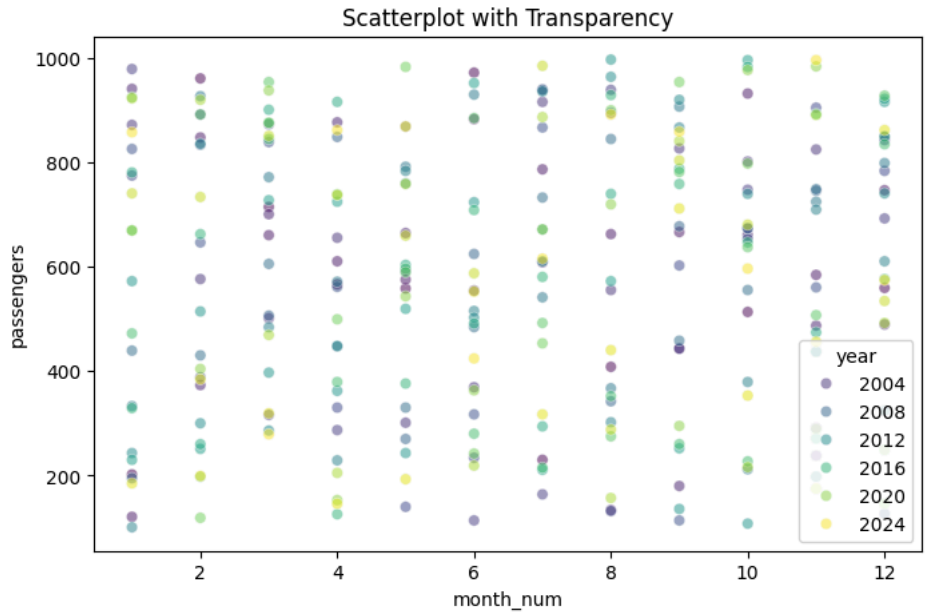


The original scatterplot shows all points clearly, but many points overlap, especially in months with multiple years of data.

Overlapping points make it hard to see the exact distribution or density of passengers.

You can notice general trends, but crowded areas hide details, so additional techniques like transparency, jittering, or hexbin plots are needed to improve clarity.

```
# 2. Scatterplot with Transparency
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x='month_num', y='passengers', hue='year', palette='viridis', alpha=0.5)
plt.title("Scatterplot with Transparency")
plt.show()
```

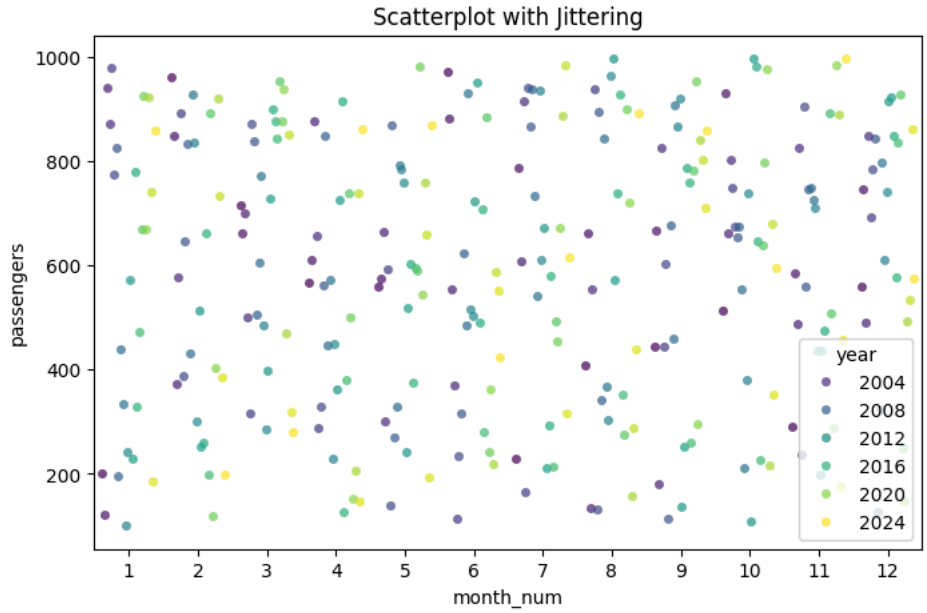


Using transparency (alpha=0.5) makes overlapping points darker, revealing where data is concentrated.

Sparse areas remain lighter, so you can visually distinguish dense vs. less dense regions.

This improves clarity compared to the original plot, making clusters and patterns across months and years easier to see.

```
# 3. Scatterplot with Jittering
plt.figure(figsize=(8,5))
sns.stripplot(data=df, x='month_num', y='passengers', hue='year',
              jitter=0.2, dodge=True, alpha=0.7, palette='viridis')
plt.title("Scatterplot with Jittering")
plt.show()
```

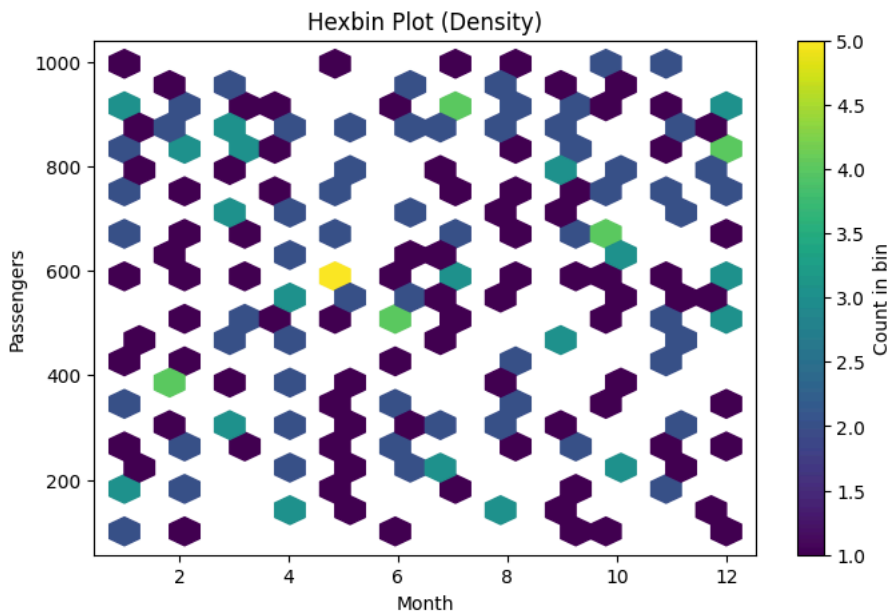


Jittering adds slight random noise to the points, separating overlapping dots on the discrete x-axis (months).

This makes it easier to see individual data points and compare passenger counts across years.

Combined with color for years, it clearly shows seasonal patterns and variations without points stacking on top of each other.

```
# 4. Hexbin Plot (density)
plt.figure(figsize=(8,5))
plt.hexbin(df['month_num'], df['passengers'], gridsize=20, cmap='viridis', mincnt=1)
plt.colorbar(label='Count in bin')
plt.xlabel("Month")
plt.ylabel("Passengers")
plt.title("Hexbin Plot (Density)")
plt.show()
```



The hexbin plot shows the density of points in hexagonal bins instead of individual dots.

Darker hexagons indicate months with more passengers, revealing crowded regions clearly.

This method is especially useful for large datasets, making seasonal peaks and yearly trends easy to spot without clutter.

vi. Visualize heatmap using Seaborn of Monthly Passenger Counts, where rows = year, columns = month (numeric or string). Interpret any recurring seasonal patterns or unusual trends over the years.

```
# Pivot data to have years as rows and months as columns
heatmap_data = df.pivot(index='year', columns='month', values='passengers')

# Optional: use month names instead of numbers
month_names = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
heatmap_data.columns = month_names

# Create heatmap
plt.figure(figsize=(12,6))
sns.heatmap(heatmap_data, annot=True, fmt="d", cmap='YlGnBu', linewidths=0.5)
plt.title("Monthly Passenger Counts Heatmap", fontsize=14)
plt.xlabel("Month")
plt.ylabel("Year")
plt.show()
```

Monthly Passenger Counts Heatmap